EX 5.6

Ayaan Arif Nadeem - 30546

EX1

Added the following implementation sbrktest.c

```c
#include "kernel/types.h"
#include "user/user.h"

void delay_loop() {
    volatile int i;
    for(i = 0; i < 500000000; i++);
}

int main() {
    printf("Before sbrk. Quickly open QEMU monitor now...\n");
    delay_loop();

    sbrk(1);

    printf("After sbrk. Quickly open QEMU monitor again...\n");
    delay_loop();

    exit(0);
}
```

Inspecting qemu monitor for user space mem shows

```
$ sbrktest
Before sbrk. Quickly open QEMU monitor now...
QEMU 8.2.2 monitor - type 'help' for more information
(qemu) info mem
vaddr            paddr            size             attr
---------------- ---------------- ---------------- -------
0000000000000000 0000000087f22000 0000000000001000 r-xu-a-
0000000000001000 0000000087f1f000 0000000000001000 rw-u---
0000000000002000 0000000087f1e000 0000000000001000 rw-----
0000000000003000 0000000087f1d000 0000000000001000 rw-u-ad
0000003fffffe000 0000000087f42000 0000000000001000 rw---ad
0000003ffffff000 0000000080006000 0000000000001000 r-x--a-
(qemu) After sbrk. Quickly open QEMU monitor again...

(qemu) info mem
vaddr            paddr            size             attr
---------------- ---------------- ---------------- -------
0000000000000000 0000000087f22000 0000000000001000 r-xu-a-
0000000000001000 0000000087f1f000 0000000000001000 rw-u---
0000000000002000 0000000087f1e000 0000000000001000 rw-----
0000000000003000 0000000087f1d000 0000000000001000 rw-u-ad
0000000000004000 0000000087f26000 0000000000001000 rw-u---
0000003fffffe000 0000000087f42000 0000000000001000 rw---ad
0000003ffffff000 0000000080006000 0000000000001000 r-x--a-
```

The kernal allocated 4096 bytes, the size column shows 0x1000 (hexadecimal for 4096). Even though sbrk(1) only requested 1 byte, the MMU maps memory in full 4096-byte page increments. The attr column shows rw-u---, meaning the PTE flags are set for Read, Write, and User accessibility.

EX2

In kalloc.c: Added an array to track references for every physical page

```
struct {
  struct spinlock lock;
  struct run *freelist;
  int ref_count[PHYSTOP / PGSIZE];
} kmem;
```

Update kfree to decrement the reference count and return early if other processes are still using the page

```
void
kfree(void *pa)
{
  struct run *r;

  if (((uint64)pa % PGSIZE) != 0 || (char *)pa < end || (uint64)pa >= PHYSTOP)
    panic("kfree");

acquire(&kmem.lock);
  int pn = (uint64)pa / PGSIZE;
  if(kmem.ref_count[pn] > 0) {
    kmem.ref_count[pn]--;
  }
  if(kmem.ref_count[pn] > 0) {
    release(&kmem.lock);
    return;
  }
  release(&kmem.lock);

  // Fill with junk to catch dangling refs.
  memset(pa, 1, PGSIZE);

  // Fill with junk to catch dangling refs.
  memset(pa, 1, PGSIZE);
```

When a new page is successfully allocated, initialize its reference count to 1.

```
void *
kalloc(void)
{
  struct run *r;

  acquire(&kmem.lock);
  r = kmem.freelist;
  if (r)
    kmem.freelist = r->next;
  release(&kmem.lock);

  if (r)
    memset((char *)r, 5, PGSIZE); // fill with junk

if(r) {
    kmem.ref_count[(uint64)r / PGSIZE] = 1;
  }

  return (void *)r;
}
```

Helper func to increase the count when fork() shares a page

```
void
kaddref(void *pa)
{
  acquire(&kmem.lock);
  kmem.ref_count[(uint64)pa / PGSIZE]++;
  release(&kmem.lock);
}
```

Define a flag for cow in riscv.h

```
#define PTE_V (1L << 0) // valid
#define PTE_R (1L << 1)
#define PTE_W (1L << 2)
#define PTE_X (1L << 3)
#define PTE_U (1L << 4) // user can access
#define PTE_COW (1L << 8)
```

Updated uvmcopy loop for cow implementation

```
int
uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)
{
  pte_t *pte;
  uint64 pa, i;
  uint flags;

  for(i = 0; i < sz; i += PGSIZE){
    if((pte = walk(old, i, 0)) == 0)
      panic("uvmcopy: pte should exist");
    if((*pte & PTE_V) == 0)
      panic("uvmcopy: page not present");

    pa = PTE2PA(*pte);
    flags = PTE_FLAGS(*pte);

// If the page is writable, clear the write bit and set COW flag
if(flags & PTE_W) {
  flags = (flags & ~PTE_W) | PTE_COW;

  // Safely update flags without corrupting the PPN
  *pte = (*pte & ~PTE_W) | PTE_COW;
}

    // Map the shared physical page into the child's page table
    if(mappages(new, i, PGSIZE, (uint64)pa, flags) != 0){
      goto err;
    }

    // Increment the reference count for the shared page
    kaddref((void*)pa);
  }
  return 0;
```

Added condition in trap.c to catch page faults

```
  } else if(r_scause() == 15) {
  // 15 is the code for Store Page Fault
  uint64 fault_va = r_stval(); // Get the virtual address that caused the fault

  // Call our copy function (we will create this next)
  if(cow_fault(p->pagetable, fault_va) < 0)
    p->killed = 1; // Kill the process if we run out of memory
} else if((which_dev = devintr()) != 0){
```

Added function in vm.c, actual implementation of cow

```
int
cow_fault(pagetable_t pagetable, uint64 va)
{
  if(va >= MAXVA) return -1;

  // Round down to the page boundary
  va = PGROUNDDOWN(va);

  pte_t *pte = walk(pagetable, va, 0);
  if(pte == 0 || (*pte & PTE_V) == 0 || (*pte & PTE_U) == 0 || (*pte & PTE_COW) == 0)
    return -1;

  uint64 pa = PTE2PA(*pte);

  // Allocate a new page
  char *mem = kalloc();
  if(mem == 0)
    return -1;

  // Copy the old page data to the new page
  memmove(mem, (char*)pa, PGSIZE);

  // Map the new page with read and write permissions
  uint flags = PTE_FLAGS(*pte);
  flags = (flags & ~PTE_COW) | PTE_W;

  // Clear the old mapping
  uvmunmap(pagetable, va, 1, 1);

// DECREMENT REFERENCE TO OLD PAGE
  kfreeref((void*)pa);

  if(mappages(pagetable, va, PGSIZE, (uint64)mem, flags) != 0){
    kfree(mem);
    return -1;
  }

  return 0;
}
```

Added check in copyout func to force a fault before the kernel writes to a COW page

```
    pte = walk(pagetable, va0, 0);
  if(pte != 0 && (*pte & PTE_V) && (*pte & PTE_U) && (*pte & PTE_COW)) {
    // Force a page allocation for the COW page
    if(cow_fault(pagetable, va0) < 0)
      return -1;
  }
```

Downloaded cowtest.c from MIT official code suite to test implementation, SUCCESS

